

Trabajo Práctico: BombLab 1C 2026

Materia: Organización del Computador (95.57/TB023)

Curso: 01 - Benitez

Fecha de entrega: 21/06/2026

Integrantes:

- Oshiro, Nadina Eimi 111.936
- Urbani, Matías 113.793

Introducción

El presente trabajo práctico está orientado a aplicar ingeniería inversa en un archivo binario que contiene un juego. Dicho juego consiste en preguntarle al jugador 6 claves aleatorias de tipo int, float y string para desactivar una bomba. Al jugador se le preguntan las claves una por una y a medida que responde correctamente pasa a la siguiente pregunta. Si alguna de las respuestas ingresadas no coincide con la clave previamente generada, el juego se termina y explota la bomba.

El objetivo es presentar un programa escrito en el lenguaje C que al ejecutarse obtenga e inyecte las claves correctas para ganar el juego. Para ello se utilizan funciones de bibliotecas del lenguaje, en especial ptrace.

Instrucciones de ejecución

El programa se compila con la línea:

```
gcc tp.c -o tp -g -Wall -Werror
```

El programa se ejecuta con la línea:

```
./tp
```

Para verificar pérdida de memoria, ejecutar con valgrind:

```
valgrind ./tp --leak-check=full -s
```

Pasos para crear el programa

Desensamblaje

Primero se desensambla el archivo binario Bomba para obtener el archivo bomba.asm desde la terminal con el comando:

```
objdump -d Bomba > bomba.asm
```

Análisis del archivo bomba.asm

Dentro del archivo bomba.asm identificamos sus secciones, prestando atención principalmente donde hay llamadas a printf, fgets, cmp y sscanf.

De esta forma, encontramos donde se guardan los valores solicitados al jugador y en el cmp el registro en el que se encuentran las claves previamente generadas de manera pseudoaleatoria con las llamadas a rand.

Clave 1 (entero)

En la línea 1730 se encuentra el primer llamado a la función printf, desde donde se espera que se imprima por pantalla la solicitud de la primera clave. Posteriormente, en las líneas 174c y 1760 se encuentran los llamados a fgets y sscanf respectivamente. En esta parte del código se procesa la solicitud de la primera clave y se extrae como dato de tipo entero.

Particularmente en la línea 176e se puede observar la instrucción de comparación entre los contenidos de r15d (la clave correcta) y de la posición 0x44 del rsp (la respuesta ingresada por el jugador).

Clave 2 (float)

La llamada a printf para solicitar la segunda clave está en la línea 1787, la llamada a fgets en la 17a2 y la de sscanf en la 17b4. Se puede observar que la validación entre la respuesta del jugador y la clave correcta está en la línea 17c8 con la instrucción ucomiss que funciona para números float.

En la instrucción mencionada, los valores comparados son los del registro xmm3 (la clave correcta) y de la posición 0x48 del rsp (respuesta recibida).

Clave 3 (entero)

La llamada a printf para solicitar la tercera clave se encuentra en la línea 17f9, mientras que las llamadas a fgets y sscanf aparecen después en 180d y 1821.

Luego, está la validación con cmp en la línea 182f entre %ebx y 0x44(%rsp) esto indica que el valor correcto está en la última modificación de ebx que es en la línea 17f5. Por esto se elige la línea 17f7 como punto de interés para poner el breakpoint y leer el valor de ebx.

Clave 4 (string)

La cuarta clave es de tipo string, nos damos cuenta observando en la validación que usa strcmp en la línea 18ae que es exclusivo para cadenas de caracteres.

Como strcmp recibe en rsi la dirección de una de las cadenas a comparar, se concluye que la clave correcta se encuentra almacenada en memoria a partir de la dirección 0x5f(%rsp).

Observando el .asm vemos que la construcción del string comienza en 0x50(%rsp) en la línea 1843 y termina en 0x5e(%rsp) en la línea 1873 por lo que se selecciona la línea 1877 para colocar el breakpoint

Clave 5 (entero)

La llamada a printf para solicitar la quinta clave está en la línea 18db, seguida por las llamadas a fgets y sscanf en las líneas 18ef y 1903.

La validación está mediante una comparación en la línea 1911 entre %ebx y 0x44(%rsp), al igual que en las otras etapas de enteros la respuesta del usuario es en 0x44 del rsi, y en rbx la clave. Como la última modificación fue en 18d7 colocamos el breakpoint en 18d9.

Clave 6 (float)

La llamada a printf para solicitar la última clave está en la línea 1929, la llamada a fgets en la 193d y la de sscanf en la 194f. La validación entre la respuesta del jugador y la clave correcta está en la línea 195f con la instrucción ucomiss.

Los valores comparados son los del registro xmm4 (la clave correcta) y de la posición 0x4c del rsp (respuesta recibida).

Programa

El programa en C sigue siempre el mismo patrón para resolver cada una de las 6 etapas:

- Se calcula la dirección real del punto de interés sumando la base de carga del proceso al offset obtenido del análisis de bomba.asm.
- Se coloca un breakpoint en esa dirección, guardando el byte original
- Se deja correr al proceso con ptrace(PTRACE_CONT) hasta que ejecuta el breakpoint, lo que genera la pausa automática del proceso, que es detectada por el proceso padre mediante waitpid.
- Con el proceso pausado, se leen sus registros con ptrace(PTRACE_GETREGS) y/o su memoria con ptrace(PTRACE_PEEKDATA) para obtener la clave correspondiente.
- Se restaura el byte original en la dirección del breakpoint y se corrige el registro rip para que vuelva a apuntar al inicio de la instrucción.
- Se inyecta la respuesta correcta escribiéndola en un pipe cuyo extremo de lectura fue conectado al stdin del proceso hijo (mediante dup2), simulando que un usuario tipeó la respuesta y presionó Enter.

La comunicación entre el programa en C y la bomba se logra mediante un pipe creado antes del fork: el extremo de lectura se redirige al stdin del proceso hijo (la bomba) con dup2, y el proceso padre escribe en el extremo de escritura cada vez que necesita inyectar una respuesta.

Es importante notar que el proceso hijo debe invocar PTRACE_TRACEME antes de ejecutar execl, ya que esto le indica al kernel que su proceso padre podrá controlarlo mediante ptrace. Inmediatamente después del execl, el kernel envía automáticamente una

señal SIGTRAP que el proceso padre espera con un primer waitpid antes de comenzar a colocar breakpoints.

Finalmente, una vez inyectadas las 6 claves, el proceso se deja correr libremente con PTRACE_CONT y el programa padre espera (en un ciclo de waitpid) a que el proceso hijo finalice, ya sea de forma normal (WIFEXITED) o por una señal (WIFSIGNALED), antes de terminar.